

Step 4 — Analysis Agent — Solution

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Basic Version

Approach 1 — the direct way

```
from pydantic import BaseModel
from langchain_openai import ChatOpenAI

class Findings(BaseModel):
    topic: str
    themes: list[str]

def run_analysis(notes):
    facts_block = "\n".join(notes.facts)
    model = ChatOpenAI(model="gpt-4o-mini").with_structured_output(Findings)
    return model.invoke(f"Group these facts into clear themes for topic {notes.topic}")
```

This works — it proves a `ResearchNotes` object can flow straight into a second specialist and come back as something structured. `themes` here is just a flat list of short strings, with no link back to which facts support which theme, and nothing checks whether every fact actually made it into a theme.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Approach 1 — nested themes, typed against Step 3's real output

```
from pydantic import BaseModel
from langchain_openai import ChatOpenAI
from agents.research_agent import ResearchNotes, run_research

class Theme(BaseModel):
    label: str
    supporting_facts: list[str]

class Findings(BaseModel):
    topic: str
    themes: list[Theme]

def run_analysis(notes: ResearchNotes) -> Findings:
    facts_block = "\n".join(f"- {fact}" for fact in notes.facts)
    model = ChatOpenAI(model="gpt-4o-mini", temperature=0).with_structured_output
    return model.invoke(
        f"Group ALL of these facts about {notes.topic} into 2-4 clear themes. "
        f"Every fact must appear under exactly one theme:\n{facts_block}"
    )

if __name__ == "__main__":
    notes = run_research("electric bikes")
    findings = run_analysis(notes)
    print(findings)
```

Difference from Basic: `Theme` is now its own nested model with a `label` and the exact `supporting_facts` behind it — a shape the Writer can actually draft from ("here's the electric-bikes-cost theme, and the 3 facts that support it"), not just a bare label. `run_analysis` takes the real `ResearchNotes` type from `research_agent.py` and the test imports and calls `run_research` directly, so this genuinely proves the two specialists' interfaces fit, rather than testing against a fresh guess at what Research's output looks like. `temperature=0` for consistent grouping. Still missing: any check that the model actually followed "every fact must appear under exactly one theme" — that instruction is a request, and the model can and sometimes does ignore part of it.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Approach 1 — coverage check, fail loudly on any dropped fact

```
from pydantic import BaseModel
from langchain_openai import ChatOpenAI
from agents.research_agent import ResearchNotes, run_research

class Theme(BaseModel):
    label: str
    supporting_facts: list[str]

class Findings(BaseModel):
    topic: str
    themes: list[Theme]

class DroppedFactsError(Exception):
    pass

def run_analysis(notes: ResearchNotes) -> Findings:
    facts_block = "\n".join(f"- {fact}" for fact in notes.facts)
    model = ChatOpenAI(model="gpt-4o-mini", temperature=0).with_structured_output
    findings = model.invoke(
        f"Group ALL of these facts about {notes.topic} into 2-4 clear themes. "
        f"Every fact must appear under exactly one theme:\n{facts_block}"
    )

    covered_facts: set[str] = set()
    for theme in findings.themes:
        covered_facts.update(theme.supporting_facts)

    missing = [fact for fact in notes.facts if fact not in covered_facts]
    if missing:
        raise DroppedFactsError(
            f"Analysis for '{notes.topic}' dropped {len(missing)} fact(s): {missi
        )
```

```

return findings

if __name__ == "__main__":
    notes = run_research("electric bikes")
    findings = run_analysis(notes)
    print(findings)

```

Expected output on a normal run: a `Findings` object printed, no exception. **Expected behavior if the model drops a fact:** a raised `DroppedFactsError` naming exactly which fact(s) got lost, instead of a `Findings` object that silently under-represents Research's actual work.

Approach 2 — one corrective follow-up instead of failing outright

```

def run_analysis(notes: ResearchNotes, retries: int = 1) -> Findings:
    facts_block = "\n".join(f"- {fact}" for fact in notes.facts)
    model = ChatOpenAI(model="gpt-4o-mini", temperature=0).with_structured_output

    instruction = (
        f"Group ALL of these facts about {notes.topic} into 2-4 clear themes. "
        f"Every fact must appear under exactly one theme:\n{facts_block}"
    )

    for attempt in range(retries + 1):
        findings = model.invoke(instruction)

        covered_facts: set[str] = set()
        for theme in findings.themes:
            covered_facts.update(theme.supporting_facts)

        missing = [fact for fact in notes.facts if fact not in covered_facts]
        if not missing:
            return findings

        instruction = (
            f"Your last grouping dropped these facts: {missing}. "
            f"Redo the grouping so every fact from the original list appears under"
        )

    raise DroppedFactsError(f"Analysis for '{notes.topic}' still dropped facts af

```

Difference from Intermediate, and between these 2 Advanced approaches: Intermediate trusts "every fact must appear under exactly one theme" as a prompt instruction and never checks whether it actually held. Approach 1 adds the check and fails fast and specifically if it didn't — cheap, and correct as a minimum bar, matching the same pattern already used in the `.env` exercise's required-

key check and Step 3's tool-call verification. Approach 2 goes further: instead of raising immediately, it gives the model one more chance, explicitly naming what it dropped, which in practice recovers most cases without ever accepting an incomplete `Findings` object as final.

Which one should you actually write? Approach 1 as the non-negotiable floor — a coverage check costs a few lines and closes a real, easy-to-hit bug (the model quietly summarizing away an inconvenient fact). Approach 2's retry is worth adding once this feeds into a pipeline where you'd rather spend one extra API call than have the whole Step 5 run fail over one dropped fact — which is exactly the trade-off Step 5's Supervisor will need to make about every specialist it calls.